Q1. Define Software Construction.

Answer:
Software Construction is the process of building software through coding, testing, verification, and debugging to produce working code.

Q2. Explain the relationship between Software Construction and Software Design.

Answer:
Construction uses design outputs, but detailed design decisions often continue during coding due to practical constraints.

Q3. What are the goals of Software Construction?

Answer:
The goals are correctness, ease of understanding, and readiness for future changes.

Q4. Why is minimizing complexity important?

Answer:
Because complex code is hard to understand, test, maintain, and increases the chance of errors.

Q5. How can complexity be minimized?

Answer:
By writing simple code, using abstraction, and applying modular design.

Q6. What is anticipating change?

Answer:
Designing software to adapt easily to future changes in requirements or technology.

Q7. Explain constructing for verification.

Answer:
Writing code that makes faults easy to detect using testing, standards, and simple structures.

Q8. Define software reuse.

Answer:
Using existing software components such as libraries or modules in new systems.

Q9. Construction for reuse vs construction with reuse.

Answer:
Construction for reuse creates reusable components, while construction with reuse uses existing ones.

Q10. What are standards in software construction?

Answer:
Rules and guidelines that improve code consistency, readability, and quality.

Q11. Construction in linear vs iterative models.

Answer:
Linear models separate construction after design, while iterative models mix design, coding, and testing.

Q12. What is construction planning?

Answer:
Defining tasks, resources, and schedules needed to build software.

Q13. What are code metrics?

Answer:
Measures like Lines of Code, Cyclomatic Complexity, and Code Churn.

Q14. What are quality metrics?

Answer:
Metrics such as defect density, test coverage, and code review results.

Q15. Why are construction languages important?

Answer:
They affect performance, reliability, maintainability, security, and portability.

Q16. Types of programming language notation.

Answer:
Linguistic, formal, and visual notation.

Q17. Coding best practices.

Answer:
Clear naming, simple control flow, error handling, security, and documentation.

Q18. Unit testing vs integration testing.

Answer:
Unit testing checks single components; integration testing checks combined components.

Q19. How to ensure construction quality?

Answer:
Through testing, static analysis, inspections, and debugging.

Q20. Phased vs incremental integration.

Answer:
Phased integrates all at once; incremental integrates step by step.